1. STUDY ON PYTHON LIBRARY FOR MACHINE LEARNING

Aim:

To study on python library for machine learning.

1. NumPy

NumPy is a fundamental numerical computing library in Python that provides support for large, multi-dimensional arrays and matrices, along with a comprehensive collection of mathematical functions. In machine learning, it is widely used for handling numerical data, performing mathematical computations and working with multi-dimensional arrays.

- Enables fast numerical computations and vectorized operations on datasets.
- Provides efficient array handling for large datasets and serves as the foundation for many ML libraries.

2. Pandas

Pandas is a high-level data analysis and manipulation library built on top of NumPy. It provides powerful data structures like DataFrame and Series that help organize, clean and process structured data efficiently for machine learning tasks.

- Simplifies data cleaning, transformation and exploratory data analysis.
- Handles missing, inconsistent and categorical data efficiently.
- Integrates seamlessly with ML and visualization libraries
- Example: Let's see an example of Pandas library.

3. Matplotlib

Matplotlib is a comprehensive data visualization library used to create static and interactive plots. In machine learning, it plays a critical role in understanding data distributions, detecting patterns and interpreting model performance through graphical representations.

Matplotlib is a visualization library that helps developers create graphs, charts, and plots for data analysis and machine learning insights.

- Helps visualize data distributions, trends and model outputs effectively.
- Supports customizable plots for analysis and result interpretation.

4. Scikit-learn

Scikit-learn is a widely used machine learning library that provides simple and efficient tools for building and evaluating machine learning models. It supports tasks such as classification, regression, clustering, preprocessing and model evaluation.

- Provides a consistent and easy-to-use API for machine learning workflows.
- Includes tools for preprocessing, model training, testing and evaluation.

5. TensorFlow

TensorFlow is an open-source deep learning framework developed by Google for building, training and deploying neural network models. It is widely used for large-scale machine learning and deep learning applications.

- Supports scalable deep learning with GPU and distributed training capabilities.
- Provides flexible APIs for designing and training neural network architectures.

6. Keras

Keras is a high-level neural network API that simplifies deep learning model development. It abstracts much of the complexity involved in building neural networks, making it especially suitable for beginners and rapid prototyping.

- Simplifies neural network creation with minimal and readable code.
- Supports fast development for regression and classification tasks.

7. PyTorch

PyTorch is an open-source deep learning library known for its dynamic computation graph, which allows models to be modified during execution. This makes PyTorch highly flexible and popular in research and experimentation.

- Supports dynamic and flexible model development
- Simplifies debugging and custom model creation
- Supports custom training logic

8. Seaborn

Seaborn is a statistical data visualization library built on Matplotlib that simplifies the creation of informative and visually appealing plots. It is widely used in machine learning and data analysis to explore patterns, relationships and distributions within datasets.

- Used for exploratory data analysis
- Simplifies statistical data visualization

- Integrates seamlessly with pandas DataFrames

9. OpenCV

OpenCV is a computer vision library used for image processing and video analysis. It enables machines to understand visual data.

This machine learning Python library is commonly used in security systems, facial recognition, autonomous vehicles, and healthcare imaging

10. XGBoost

XGBoost is one of the most powerful gradient boosting libraries used for supervised learning problems. It is highly efficient and widely used in competitive machine learning projects.

2. a) simple Graph

Coding:

```
# importing the required module
import matplotlib.pyplot as plt
 # x axis values
x = [1,2,3]
# corresponding y axis values
y = [2,4,1]

# plotting the points
plt.plot(x, y)

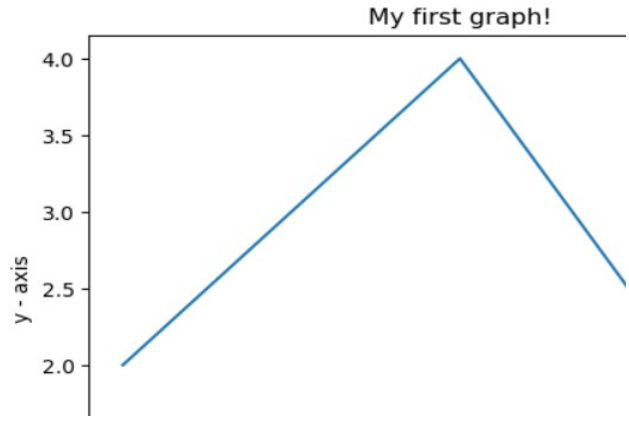
# naming the x axis
plt.xlabel('x - axis')
# naming the y axis
plt.ylabel('y - axis')

# giving a title to my graph
plt.title('My first graph!')

# function to show the plot
plt.show()
```

Output:-

```
# function to show the plot
plt.show()
```



2. b) Mean, Median, Mode, Variance, Standard Deviation

Coding :

```
import statistics
list = [15, 25, 35, 45, 60, 50, 45, 50]
print ("The average of list values is : ",end="")
print (statistics.mean(list))
print ("The Mode of list values is : ",end="")
print (statistics.mode(list))
print ("The Median of list values is : ",end="")
print (statistics.median(list))
print ("The standard deviation of list values is : ",end="")
print (statistics.stdev(list))
```

output :

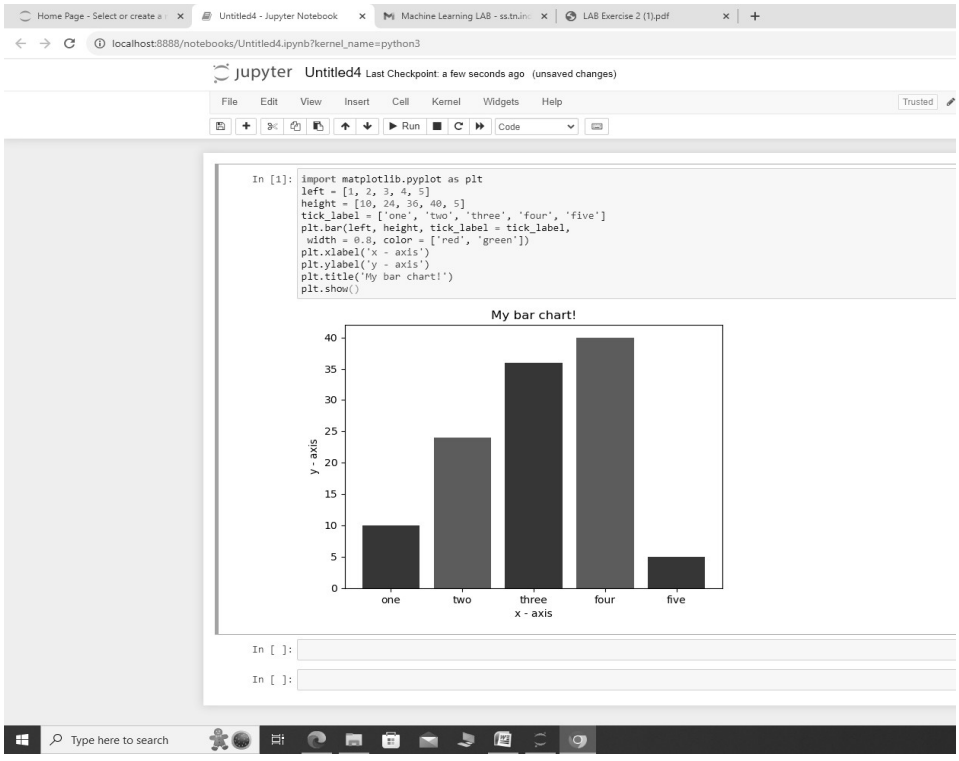
```
The average of list values is : 40.625
The Mode of list values is : 45
The Median of list values is : 45.0
The standard deviation of list values is : 14.74485382381644
```

2. c) Bar Chart using Python

Coding :

```
import matplotlib.pyplot as plt
left = [1, 2, 3, 4, 5]
height = [10, 24, 36, 40, 5]
tick_label = ['one', 'two', 'three', 'four', 'five']
plt.bar(left, height, tick_label = tick_label,
 width = 0.8, color = ['red', 'green'])
plt.xlabel('x - axis')
plt.ylabel('y - axis')
plt.title('My bar chart!')
plt.show()
```

Output:-

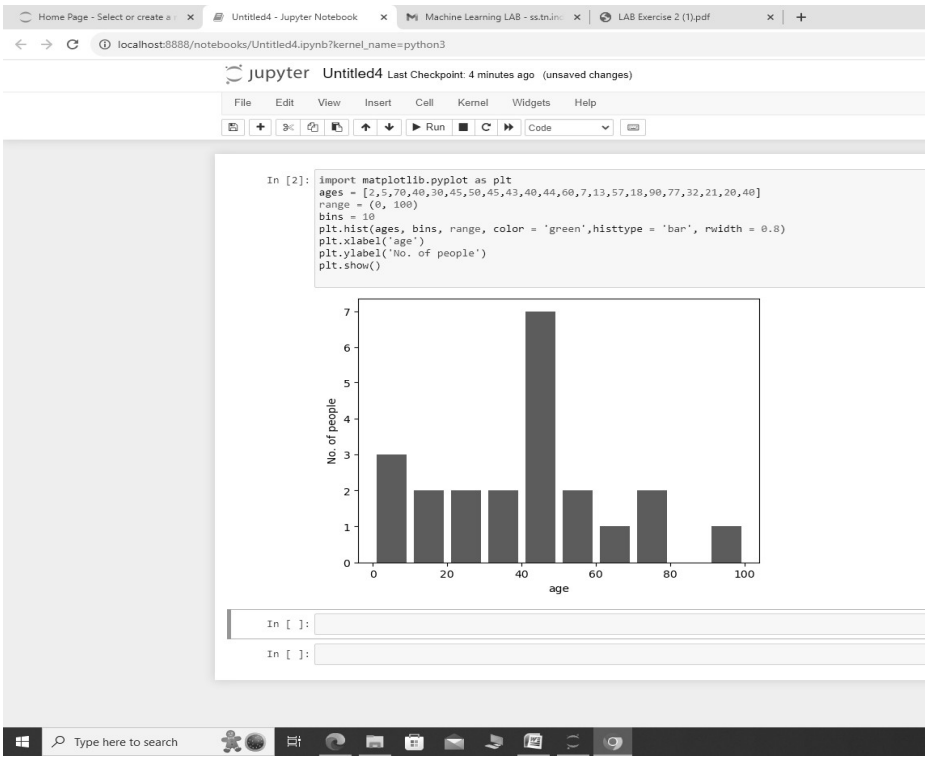


2 d) Histogram

Coding :

```
import matplotlib.pyplot as plt
ages = [2,5,70,40,30,45,50,45,43,40,44,60,7,13,57,18,90,77,32,21,20,40]
range = (0, 100)
bins = 10
plt.hist(ages, bins, range, color = 'green',histtype = 'bar', rwidth = 0.8)
plt.xlabel('age')
plt.ylabel('No. of people')
plt.show()
```

Output :

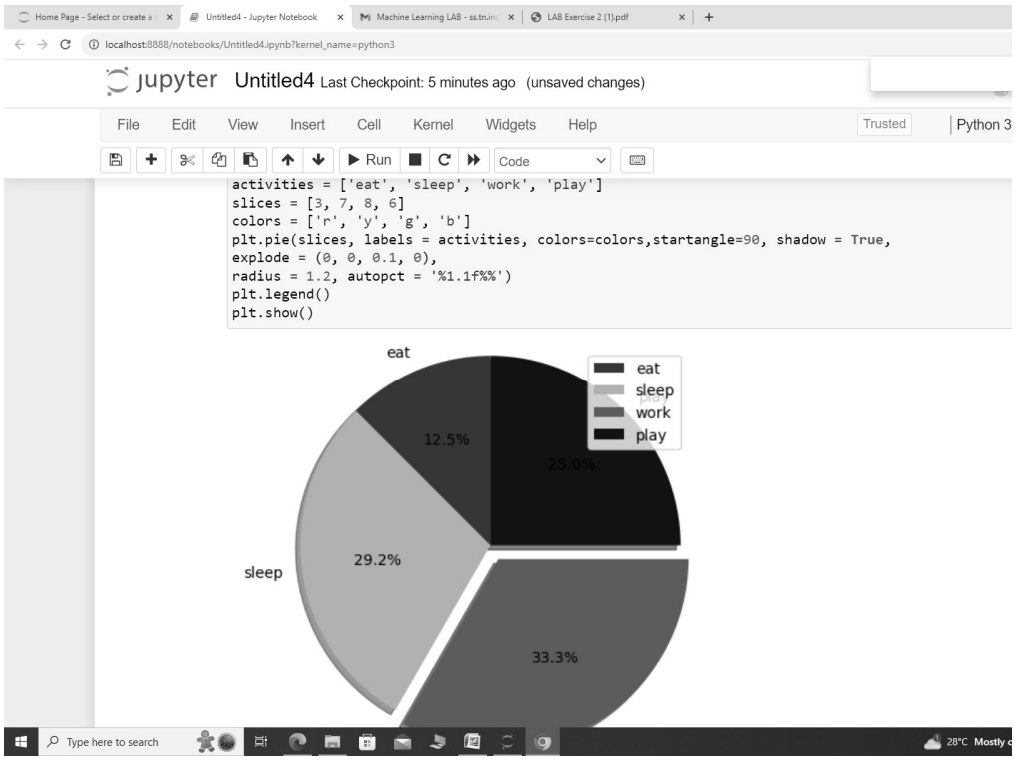


2 e) Pie-chart

Coding :

```
import matplotlib.pyplot as plt
activities = ['eat', 'sleep', 'work', 'play']
slices = [3, 7, 8, 6]
colors = ['r', 'y', 'g', 'b']
plt.pie(slices, labels = activities, colors=colors,startangle=90, shadow = True,
explode = (0, 0, 0.1, 0),
radius = 1.2, autopct = '%1.1f%%')
plt.legend()
plt.show()
```

Output :

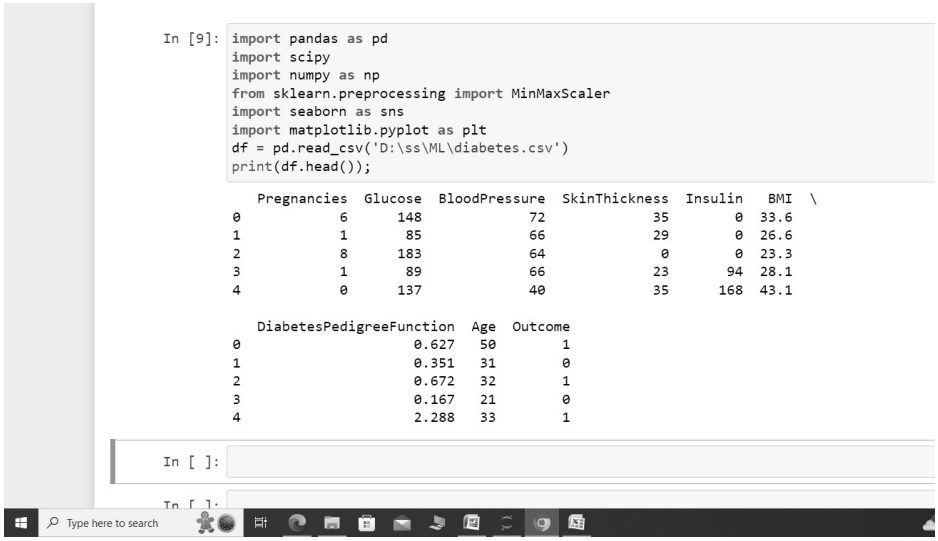


3. Data Preprocessing Technique

Coding:

```
import pandas as pd
import scipy
import numpy as np
from sklearn.preprocessing import MinMaxScaler
import seaborn as sns
import matplotlib.pyplot as plt
df = pd.read_csv('D:\ss\ML\diabetes.csv')
print(df.head());
```

Output :

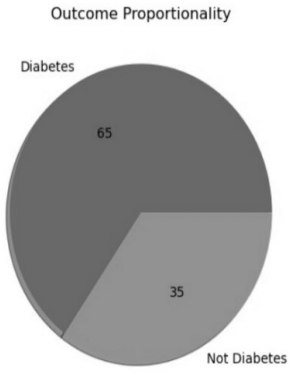


```
2                    0.672   32        1
3                    0.167   21        0
4                    2.288   33        1
```

In [10]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               768 non-null    int64
 1   Glucose                   768 non-null    int64
 2   BloodPressure             768 non-null    int64
 3   SkinThickness             768 non-null    int64
 4   Insulin                   768 non-null    int64
 5   BMI                       768 non-null    float64
```

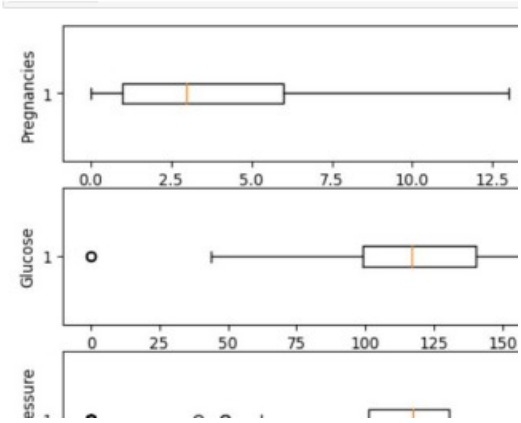
In [7]:
```
plt.pie(df.Outcome.value_counts(),
        labels= ['Diabetes', 'Not Diabetes'],
        autopct='%.f', shadow=True)
plt.title('Outcome Proportionality')
plt.show()
```



In [4]:
```
df.isnull().sum()
```

Out[4]:
```
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64
```

```
# Box Plots
fig, axs = plt.subplots(9,1,dpi=95, figsize=(7,17))
i = 0
for col in df.columns:
    axs[i].boxplot(df[col], vert=False)
    axs[i].set_ylabel(col)
    i+=1
plt.show()
```



4. Linear Regression algorithms (Supervised Learning Model)

Coding :

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

iris = datasets.load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

log_reg = LogisticRegression()
log_reg.fit(X_train, y_train)

y_pred = log_reg.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

output :

Accuracy: 1.0

5. Logistic Regression for classification

Program:

```
import numpy
from sklearn import linear_model

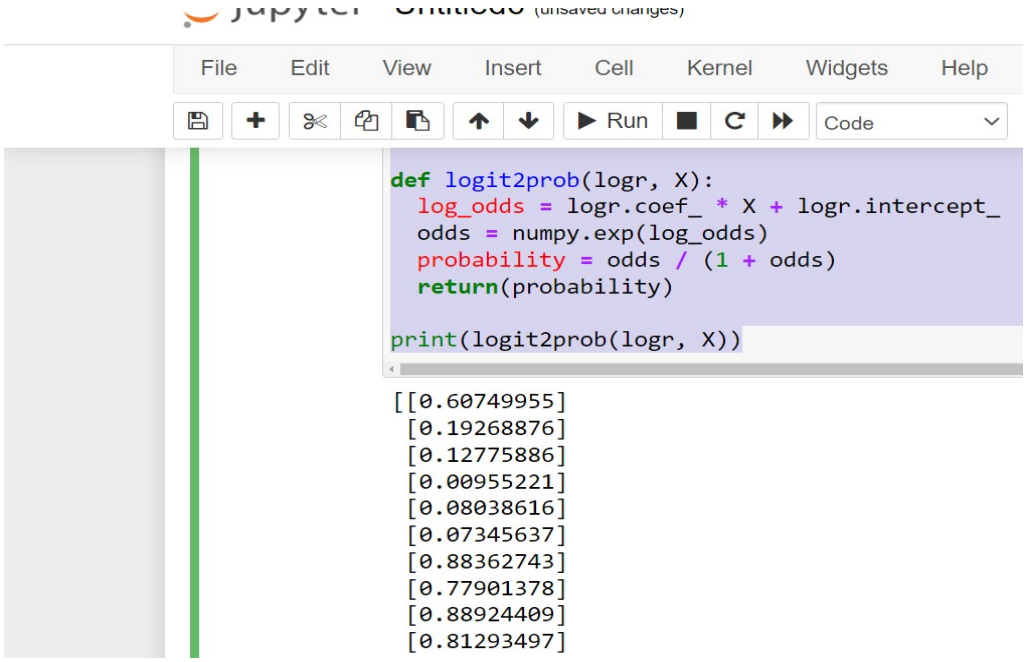
X = numpy.array([3.78, 2.44, 2.09, 0.14, 1.72, 1.65, 4.92, 4.37, 4.96, 4.52,
3.69, 5.88]).reshape(-1,1)
y = numpy.array([0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1])

logr = linear_model.LogisticRegression()
logr.fit(X,y)

def logit2prob(logr, X):
  log_odds = logr.coef_ * X + logr.intercept_
  odds = numpy.exp(log_odds)
  probability = odds / (1 + odds)
  return(probability)

print(logit2prob(logr, X))
```

Output :



6. K-Means_Clustering

Program:

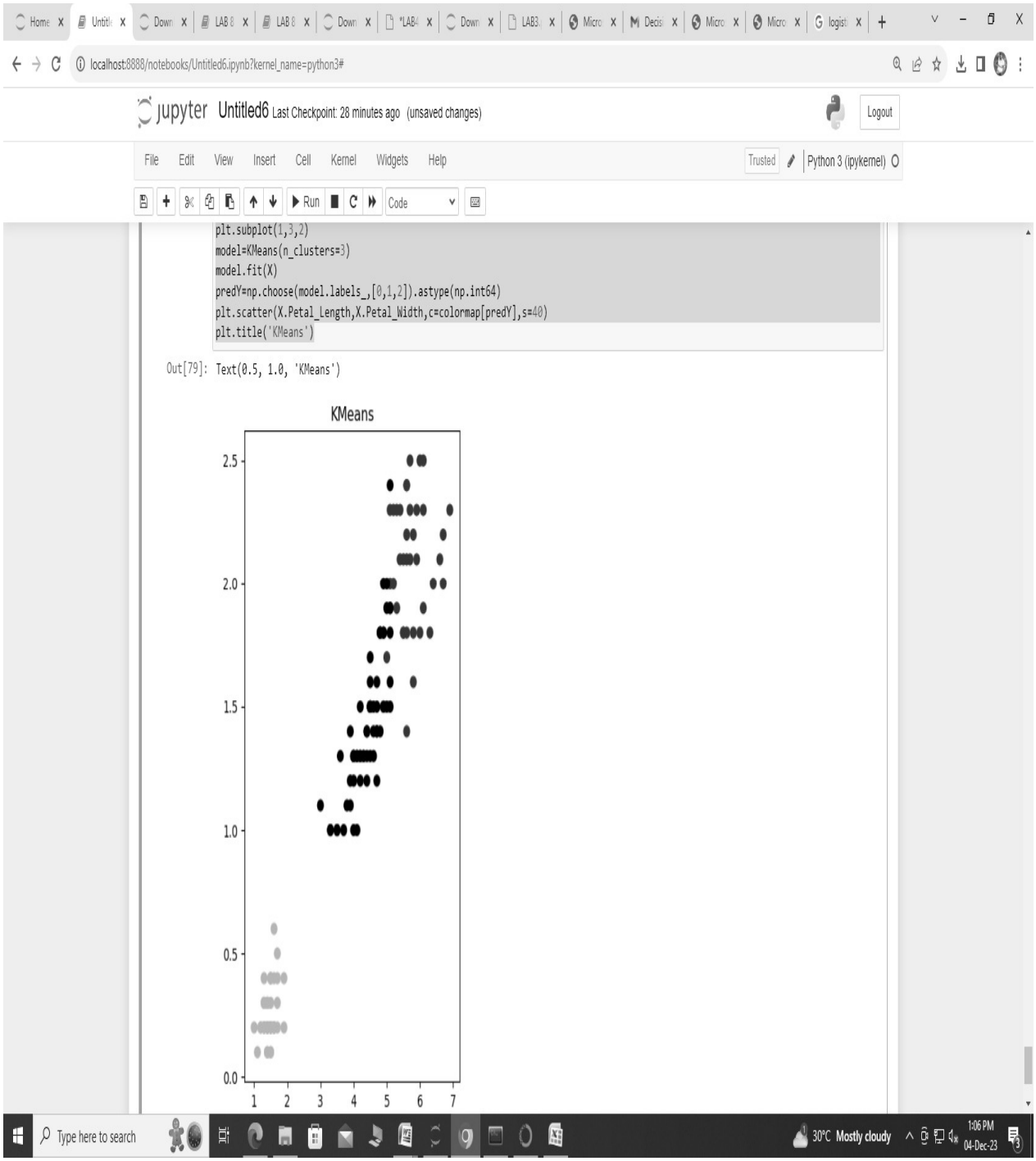
```
from sklearn.cluster import KMeans
from sklearn import preprocessing
from sklearn.mixture import GaussianMixture
from sklearn.datasets import load_iris
import sklearn.metrics as sm
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
dataset=load_iris()

# print(dataset)
X=pd.DataFrame(dataset.data)
X.columns=['Sepal_Length','Sepal_Width','Petal_Length','Petal_Width']
y=pd.DataFrame(dataset.target)
y.columns=['Targets']

# print(X)
plt.figure(figsize=(14,7))
colormap=np.array(['red','lime','black'])

# K-PLOT
plt.subplot(1,3,2)
model=KMeans(n_clusters=3)
model.fit(X)
predY=np.choose(model.labels_,[0,1,2]).astype(np.int64)
plt.scatter(X.Petal_Length,X.Petal_Width,c=colormap[predY],s=40)
plt.title('KMeans')
```

Output :


localhost:8888/notebooks/Untitled6.ipynb?kernel_name=python3#
jupyter Untitled6 Last Checkpoint: 28 minutes ago (unsaved changes)
Logout
File Edit View Insert Cell Kernel Widgets Help
Trusted
Python 3 (ipykernel)
Run
Code
plt.subplot(1,3,2)
model=KMeans(n_clusters=3)
model.fit(X)
predY=np.choose(model.labels_,[0,1,2]).astype(np.int64)
plt.scatter(X.Petal_Length,X.Petal_Width,c=colormap[predY],s=40)
plt.title('KMeans')
Out[79]: Text(0.5, 1.0, 'KMeans')
KMeans
2.5
2.0
1.5
1.0
0.5
0.0
1
2
3
4
5
6
7
Type here to search
30°C Mostly cloudy
1:06 PM
04-Dec-23

7.Naïve Bayes Classifier Algorithm

Data set: tennisplay.csv

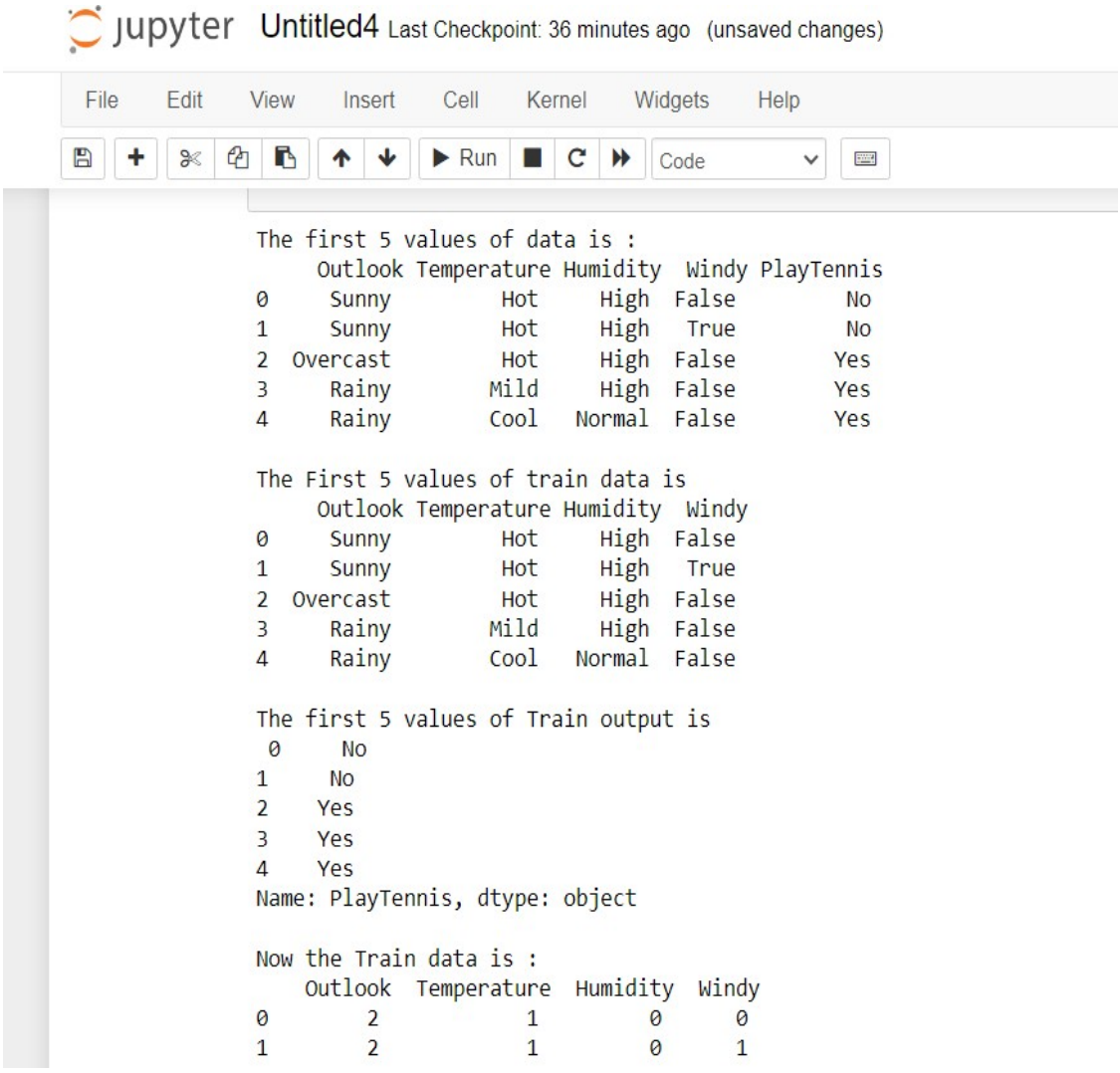
Outlook	Temperature	Humidity	Windy	PlayTennis
Sunny	Hot	High	FALSE	No
Sunny	Hot	High	TRUE	No
Overcast	Hot	High	FALSE	Yes
Rainy	Mild	High	FALSE	Yes
Rainy	Cool	Normal	FALSE	Yes
Rainy	Cool	Normal	TRUE	No
Overcast	Cool	Normal	TRUE	Yes
Sunny	Mild	High	FALSE	No
Sunny	Cool	Normal	FALSE	Yes
Rainy	Mild	Normal	FALSE	Yes
Sunny	Mild	Normal	TRUE	Yes
Overcast	Mild	High	TRUE	Yes
Overcast	Hot	Normal	FALSE	Yes
Rainy	Mild	High	TRUE	No

Coding :

```
import pandas as pd
from sklearn import tree
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
data = pd.read_csv('d:/ss/ML/tennisplay.csv')
print("The first 5 values of data is :\n",data.head())
X = data.iloc[:,:-1]
print("\nThe First 5 values of train data is\n",X.head())
y = data.iloc[:,-1]
print("\nThe first 5 values of Train output is\n",y.head())
le_outlook = LabelEncoder()
X.Outlook = le_outlook.fit_transform(X.Outlook)
le_Temperature = LabelEncoder()
X.Temperature = le_Temperature.fit_transform(X.Temperature)
le_Humidity = LabelEncoder()
X.Humidity = le_Humidity.fit_transform(X.Humidity)
le_Windy = LabelEncoder()
X.Windy = le_Windy.fit_transform(X.Windy)
```

```
print("\nNow the Train data is :\n",X.head())
le_PlayTennis = LabelEncoder()
y = le_PlayTennis.fit_transform(y)
print("\nNow the Train output is\n",y)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.20)
classifier = GaussianNB()
classifier.fit(X_train,y_train)
from sklearn.metrics import accuracy_score
print("Accuracy is:",accuracy_score ,classifier.predict(X_test),y_test))
```

Output:-




```
The first 5 values of data is :
     Outlook Temperature Humidity  Windy PlayTennis
0      Sunny         Hot     High  False         No
1      Sunny         Hot     High   True         No
2   Overcast         Hot     High  False        Yes
3      Rainy        Mild     High  False        Yes
4      Rainy        Cool   Normal  False        Yes

The First 5 values of train data is
     Outlook Temperature Humidity  Windy
0      Sunny         Hot     High  False
1      Sunny         Hot     High   True
2   Overcast         Hot     High  False
3      Rainy        Mild     High  False
4      Rainy        Cool   Normal  False

The first 5 values of Train output is
 0     No
1     No
2    Yes
3    Yes
4    Yes
Name: PlayTennis, dtype: object

Now the Train data is :
    Outlook  Temperature  Humidity  Windy
0        2            1         0      0
1        2            1         0      1
```

8. Support Vector Classification

Program:

```
#Importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

#Importing the dataset
dataset = pd.read_csv('d:/ss/ML/Social_Network_Ads.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values

#Splitting the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25,
random_state = 0)
#Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

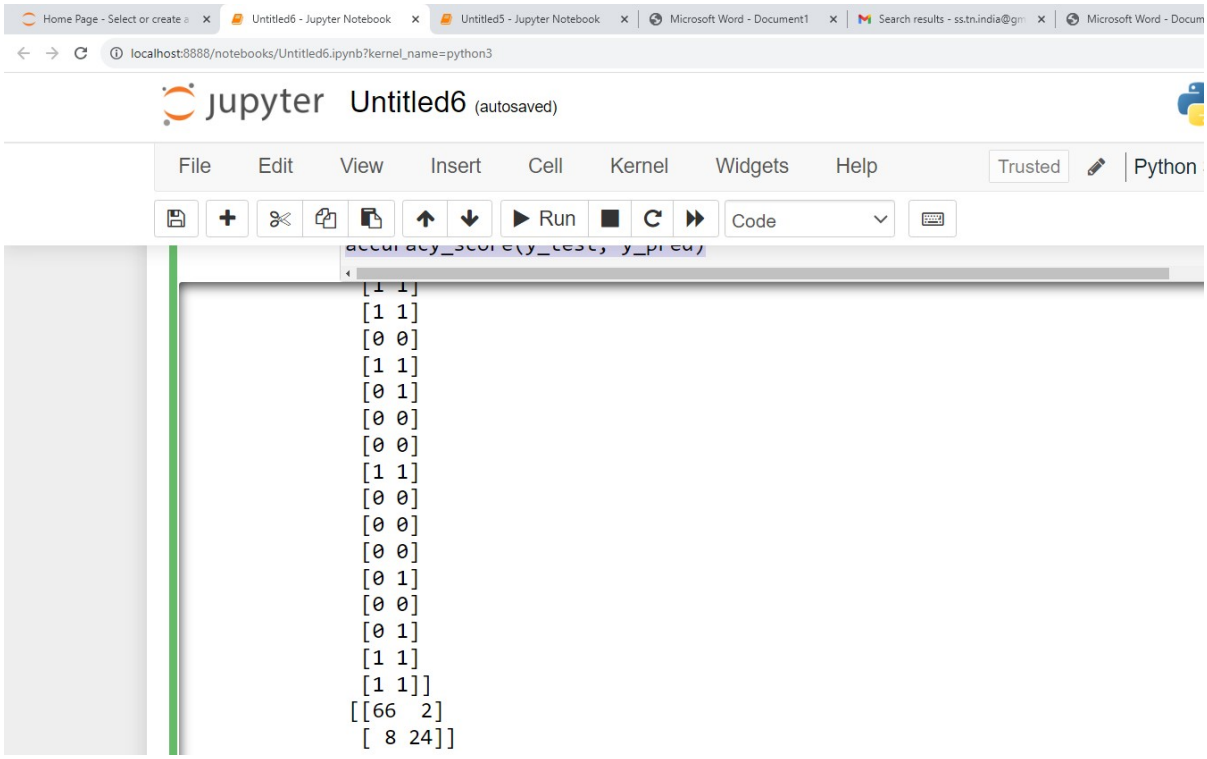
#Training the SVM model on the Training set
from sklearn.svm import SVC
classifier = SVC(kernel = 'linear', random_state = 0)
classifier.fit(X_train, y_train)

#Predicting a new result
print(classifier.predict(sc.transform([[30,87000]])))

#Predicting the Test set results
y_pred = classifier.predict(X_test)
print(np.concatenate((y_pred.reshape(len(y_pred),1),
y_test.reshape(len(y_test),1)),1))

#Making the Confusion Matrix
from sklearn.metrics import confusion_matrix, accuracy_score
cm = confusion_matrix(y_test, y_pred)
print(cm)
accuracy_score(y_test, y_pred)
```

output :-


Home Page - Select or create
Untitled6 - Jupyter Notebook
Untitled5 - Jupyter Notebook
Microsoft Word - Document1
Search results - ss.tn.india@gm
Microsoft Word - Docum
localhost:8888/notebooks/Untitled6.ipynb?kernel_name=python3
jupyter Untitled6 (autosaved)
File Edit View Insert Cell Kernel Widgets Help
Trusted
Python
Run
Code
[1 1]
[1 1]
[0 0]
[1 1]
[0 1]
[0 0]
[0 0]
[1 1]
[0 0]
[0 0]
[0 0]
[0 1]
[0 0]
[0 1]
[1 1]
[1 1]]
[[66 2]
[8 24]]

9. Bayes Network for Medical Data Analysis

Some instance from the dataset:

age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope
63	1	1	145	233	1	2	150	0	2.3	3
67	1	4	160	286	0	2	108	1	1.5	2
67	1	4	120	229	0	2	129	1	2.6	2
41	0	2	130	204	0	2	172	0	1.4	1

Program:

```
import numpy as np
import csv
import pandas as pd
from pgmpy.models import BayesianNetwork
from pgmpy.estimators import MaximumLikelihoodEstimator
from pgmpy.inference import VariableElimination
#read Cleveland Heart Disease data
heartDisease = pd.read_csv('d://ss/ML/heart.csv')
heartDisease = heartDisease.replace('?',np.nan)

#display the data
print('Few examples from the dataset are given below')
print(heartDisease.head())

#Model Bayesian Network
Model=BayesianNetwork([('age','trestbps'),('age','fbs'),('sex','trestbps'),('exang'
,'trestbps'),('trestbps','heartdisease'),('fbs','heartdisease'),('heartdisease','reste
cg'),('heartdisease','thalach'),('heartdisease','chol')])

#Learning CPDs using Maximum Likelihood Estimators
print('\n Learning CPD using Maximum likelihood estimators')
model.fit(heartDisease,estimator=MaximumLikelihoodEstimator)

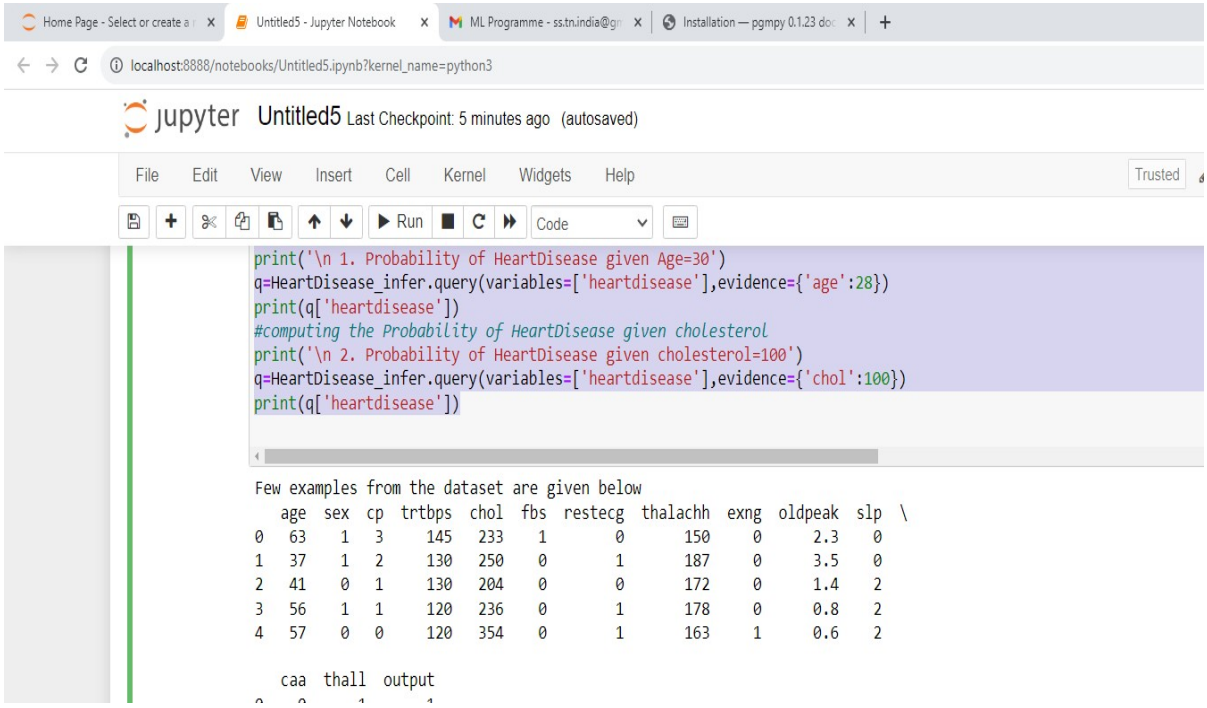
# Inferencing with Bayesian Network
print('\n Inferencing with Bayesian Network:')
```

```
HeartDisease_infer = VariableElimination(model)

#computing the Probability of HeartDisease given Age
print('\n 1. Probability of HeartDisease given Age=30')
q=HeartDisease_infer.query(variables=['heartdisease'],evidence={'age':28})
print(q['heartdisease'])

#computing the Probability of HeartDisease given cholesterol
print('\n 2. Probability of HeartDisease given cholesterol=100')
q=HeartDisease_infer.query(variables=['heartdisease'],evidence={'chol':100})
print(q['heartdisease'])
```

Output :-



Learning CPD using Maximum likelihood estimators

Inferencing with Bayesian Network:

1. Probability of HeartDisease given Age=28

heartdisease	phi(heartdisease)
heartdisease_0	0.6791
heartdisease_1	0.1212
heartdisease_2	0.0810
heartdisease_3	0.0939
heartdisease_4	0.0247

2. Probability of HeartDisease given cholesterol=100

heartdisease	phi(heartdisease)
heartdisease_0	0.5400
heartdisease_1	0.1533
heartdisease_2	0.1303
heartdisease_3	0.1259
heartdisease_4	0.0506

10. K-Nearest Neighbor algorithm (KNN)

Program:

```
#importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

#Importing the dataset
dataset = pd.read_csv('d:/ss/ML/Social_Network_Ads.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values

#Splitting the dataset into the Training set and Test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25,
random_state = 0)

#Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

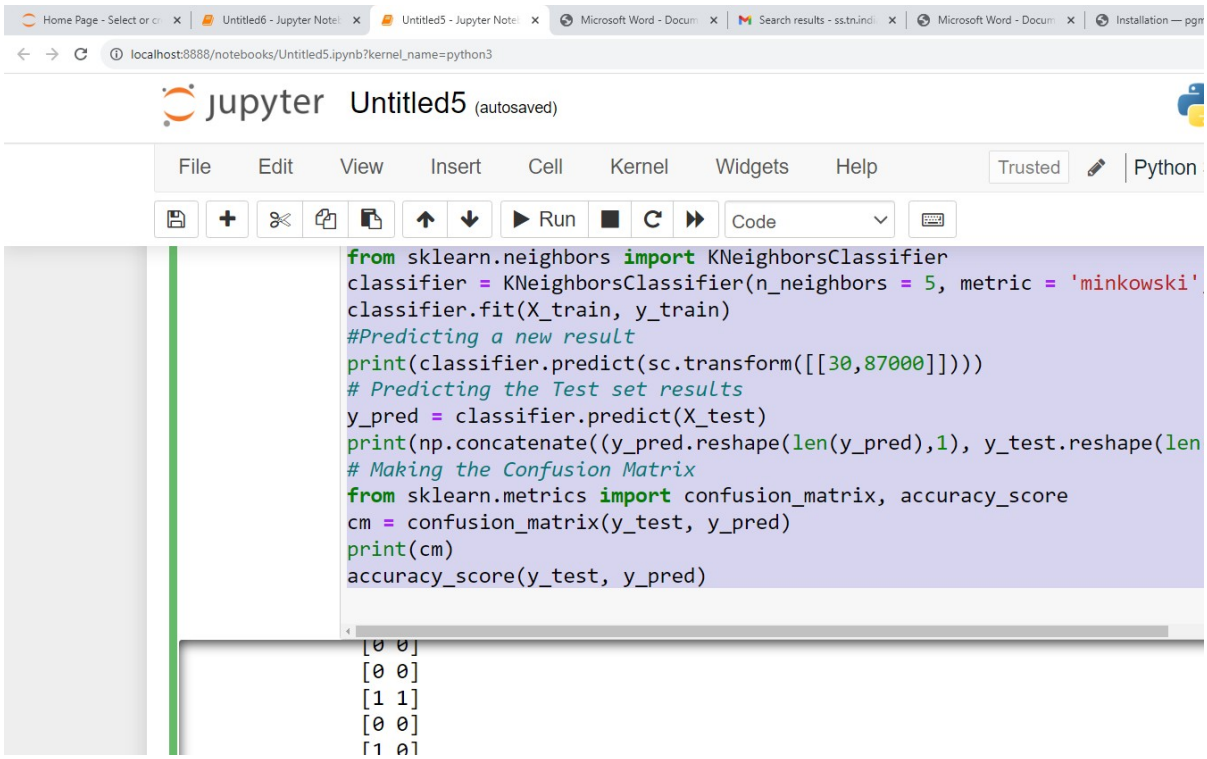
#Training the K-NN model on the Training set
from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p = 2)
classifier.fit(X_train, y_train)

#Predicting a new result
print(classifier.predict(sc.transform([[30,87000]])))

# Predicting the Test set results
y_pred = classifier.predict(X_test)
print(np.concatenate((y_pred.reshape(len(y_pred),1),
y_test.reshape(len(y_test),1)),1))
```

```
# Making the Confusion Matrix
from sklearn.metrics import confusion_matrix, accuracy_score
cm = confusion_matrix(y_test, y_pred)
print(cm)
accuracy_score(y_test, y_pred)
```

Output:



Accuracy
[[64 4]
[3 29]]

Accuracy 0.93

11. Decision Tree Classifier

Program:

```
#importing the libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

#importing the data set
dataset = pd.read_csv('D:/ss/ML/Social_Network_Ads.csv')
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values

#splitting the dataset into train set and test set
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25,
random_state = 0)

#Feature Scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

#Training the Decision Tree Classification model on the Training set
from sklearn.tree import DecisionTreeClassifier

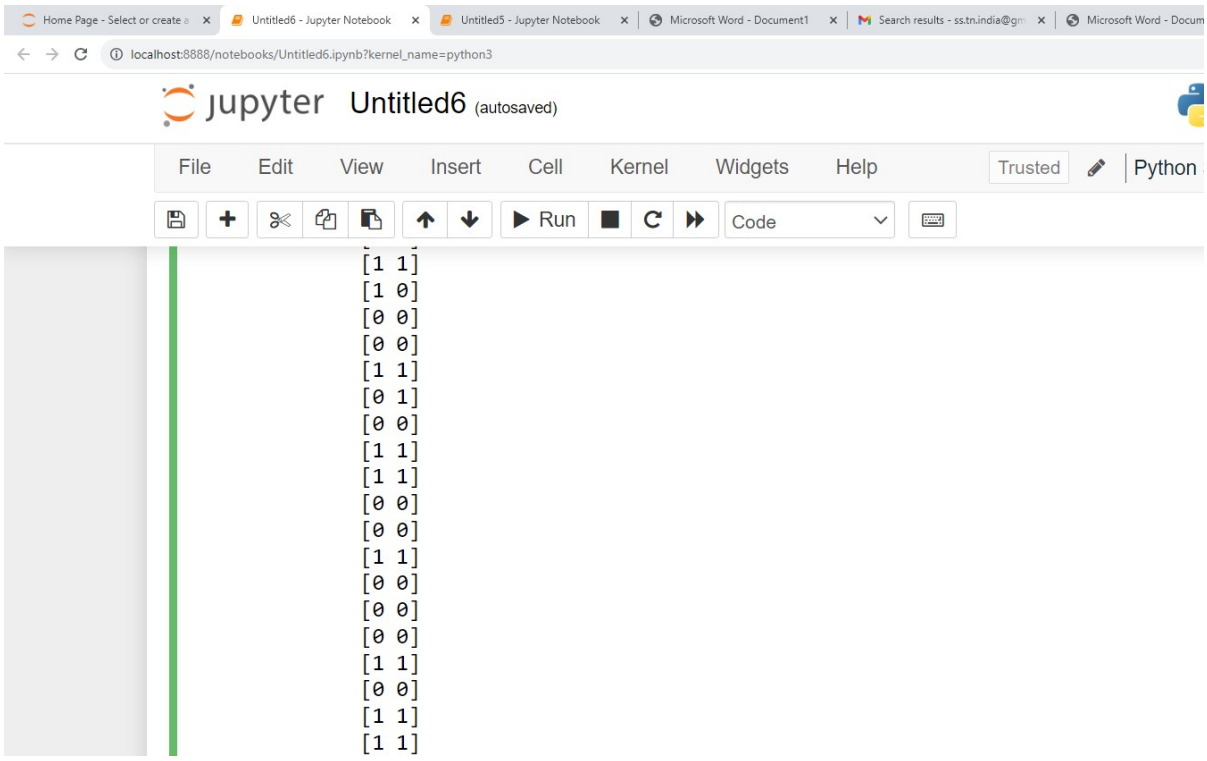
classifier = DecisionTreeClassifier(criterion = 'entropy', random_state = 0)
classifier.fit(X_train, y_train)

#Predicting a new result
print(classifier.predict(sc.transform([[30,87000]])))

#Predicting the Test set results
y_pred = classifier.predict(X_test)
print(np.concatenate((y_pred.reshape(len(y_pred),1),
y_test.reshape(len(y_test),1)),1))
```

```
#Making the Confusion Matrix
from sklearn.metrics import confusion_matrix, accuracy_score
cm = confusion_matrix(y_test, y_pred)
print(cm)
accuracy_score(y_test, y_pred)
```

Output:-



12. Decision tree based ID3 algorithm

Program :

```
import numpy as np
import math
import csv

def read_data(filename):
    with open(filename, 'r') as csvfile:
        datareader = csv.reader(csvfile, delimiter=',')
        headers = next(datareader)
        metadata = []
        traindata = []
        for name in headers:
            metadata.append(name)
        for row in datareader:
            traindata.append(row)

    return (metadata, traindata)

class Node:
    def __init__(self, attribute):
        self.attribute = attribute
        self.children = []
        self.answer = ""

    def __str__(self):
        return self.attribute

def subtables(data, col, delete):
    dict = {}
    items = np.unique(data[:, col])
    count = np.zeros((items.shape[0], 1), dtype=np.int32)

    for x in range(items.shape[0]):
        for y in range(data.shape[0]):
            if data[y, col] == items[x]:
                count[x] += 1

    for x in range(items.shape[0]):
        dict[items[x]] = np.empty((int(count[x]), data.shape[1]), dtype="|S32")
        pos = 0
        for y in range(data.shape[0]):
```

```
            if data[y, col] == items[x]:
                dict[items[x]][pos] = data[y]
                pos += 1
        if delete:
            dict[items[x]] = np.delete(dict[items[x]], col, 1)

    return items, dict

def entropy(S):
    items = np.unique(S)

    if items.size == 1:
        return 0

    counts = np.zeros((items.shape[0], 1))
    sums = 0

    for x in range(items.shape[0]):
        counts[x] = sum(S == items[x]) / (S.size * 1.0)

    for count in counts:
        sums += -1 * count * math.log(count, 2)
    return sums

def gain_ratio(data, col):
    items, dict = subtables(data, col, delete=False)

    total_size = data.shape[0]
    entropies = np.zeros((items.shape[0], 1))
    intrinsic = np.zeros((items.shape[0], 1))

    for x in range(items.shape[0]):
        ratio = dict[items[x]].shape[0]/(total_size * 1.0)
        entropies[x] = ratio * entropy(dict[items[x]][:, -1])
        intrinsic[x] = ratio * math.log(ratio, 2)

    total_entropy = entropy(data[:, -1])
    iv = -1 * sum(intrinsic)

    for x in range(entropies.shape[0]):
        total_entropy -= entropies[x]

    return total_entropy / iv

def create_node(data, metadata):
```

```
    if (np.unique(data[:, -1])).shape[0] == 1:
        node = Node("")
        node.answer = np.unique(data[:, -1])[0]
        return node

    gains = np.zeros((data.shape[1] - 1, 1))

    for col in range(data.shape[1] - 1):
        gains[col] = gain_ratio(data, col)

    split = np.argmax(gains)

    node = Node(metadata[split])
    metadata = np.delete(metadata, split, 0)

    items, dict = subtables(data, split, delete=True)

    for x in range(items.shape[0]):
        child = create_node(dict[items[x]], metadata)
        node.children.append((items[x], child))

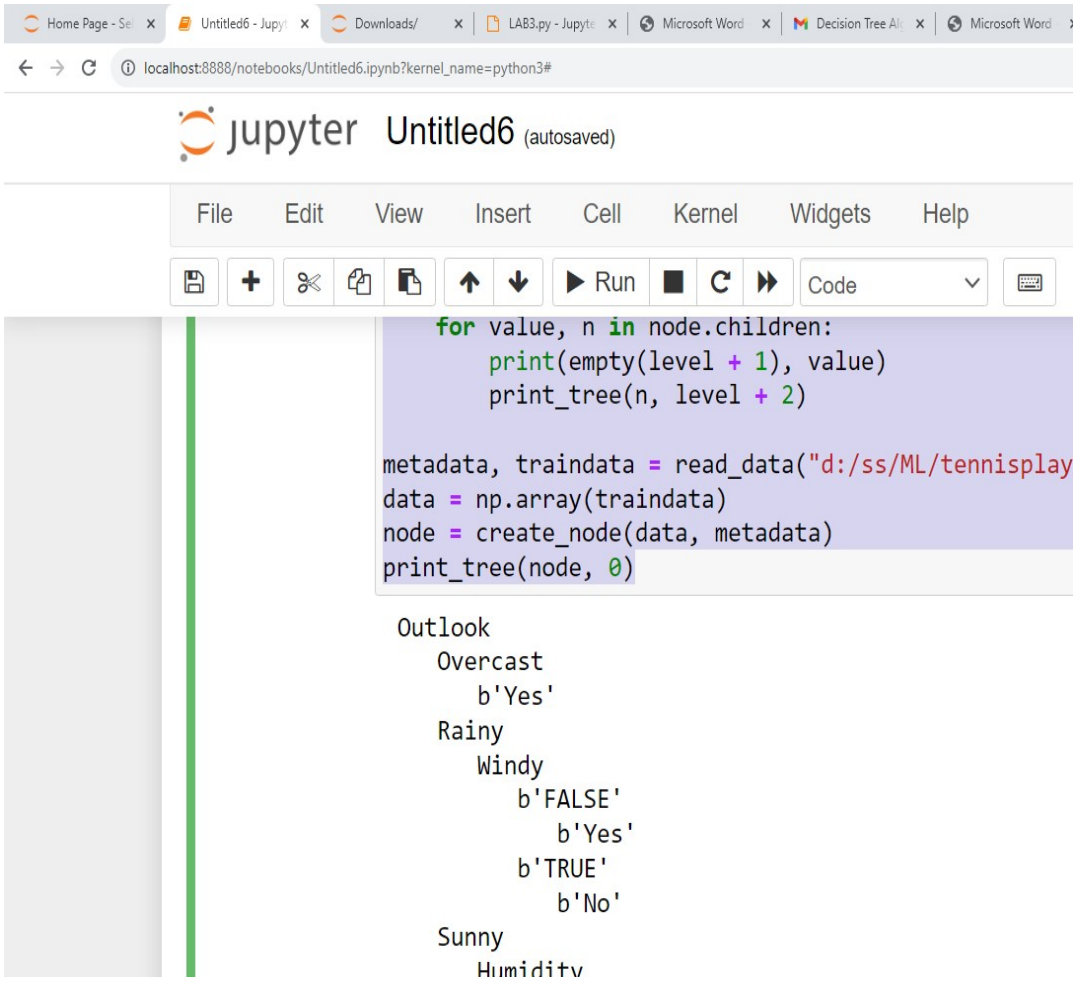
    return node

def empty(size):
    s = ""
    for x in range(size):
        s += "   "
    return s

def print_tree(node, level):
    if node.answer != "":
        print(empty(level), node.answer)
        return
    print(empty(level), node.attribute)
    for value, n in node.children:
        print(empty(level + 1), value)
        print_tree(n, level + 2)

metadata, traindata = read_data("d:/ss/ML/tennisplay.csv")
data = np.array(traindata)
node = create_node(data, metadata)
print_tree(node, 0)
```

Output:


Home Page - Sel
Untitled6 - Jupyt
Downloads/
LAB3.py - Jupyte
Microsoft Word
Decision Tree Alg
Microsoft Word
localhost:8888/notebooks/Untitled6.ipynb?kernel_name=python3#
jupyter Untitled6 (autosaved)
File Edit View Insert Cell Kernel Widgets Help
Run
Code
for value, n in node.children:
print(empty(level + 1), value)
print_tree(n, level + 2)
metadata, traindata = read_data("d:/ss/ML/tennisplay
data = np.array(traindata)
node = create_node(data, metadata)
print_tree(node, 0)
Outlook
Overcast
b'Yes'
Rainy
Windy
b'FALSE'
b'Yes'
b'TRUE'
b'No'
Sunny
Humidity

13. Artificial Neural Network

Program :

```
import numpy as np
X = np.array(([2, 9], [1, 5], [3, 6]), dtype=float)
y = np.array(([92], [86], [89]), dtype=float)
X = X/np.amax(X, axis=0)       # maximum of X array
y = y/100                      # max test score is 100
class Neural_Network(object):
    def __init__(self):
    # Parameters
        self.inputSize = 2
        self.outputSize = 1
        self.hiddenSize = 3
    # Weights
        self.W1 = np.random.randn(self.inputSize, self.hiddenSize)
    # (3x2) weight matrix from input to hidden layer
        self.W2 = np.random.randn(self.hiddenSize, self.outputSize)
     # (3x1) weight matrix from hidden to output layer

    def forward(self, X):
    #forward propagation through our network
        self.z = np.dot(X, self.W1)
            # dot product of X (input) and first set of 3x2 weights

  self.z2 = self.sigmoid(self.z)           # activation function
        self.z3 = np.dot(self.z2, self.W2)

       # dot product of hidden layer (z2) and second set of 3x1 weights
        o = self.sigmoid(self.z3)                # final activation function
        return o

    def sigmoid(self, s):
        return 1/(1+np.exp(-s))     # activation function

    def sigmoidPrime(self, s):
        return s * (1 - s)         # derivative of sigmoid
        def backward(self, X, y, o):
```

```
  # backward propgate through the network
        self.o_error = y - o        # error in output
        self.o_delta = self.o_error*self.sigmoidPrime(o)
        self.z2_error = self.o_delta.dot(self.W2.T)
        self.z2_delta = self.z2_error*self.sigmoidPrime(self.z2)
        self.W1 += X.T.dot(self.z2_delta)
        self.W2 += self.z2.T.dot(self.o_delta)
    def train (self, X, y):
        o = self.forward(X)
        self.backward(X, y, o)

NN = Neural_Network()
print ("\nInput: \n" + str(X))
print ("\nActual Output: \n" + str(y))
print ("\nPredicted Output: \n" + str(NN.forward(X)))
print ("\nLoss: \n" + str(np.mean(np.square(y - NN.forward(X))))     # mean sum
squared loss)
NN.train(X, y)
```

Output:-

